Virtual Memory Management

- Swapping -

Youngjae Kim (Ph.D.)

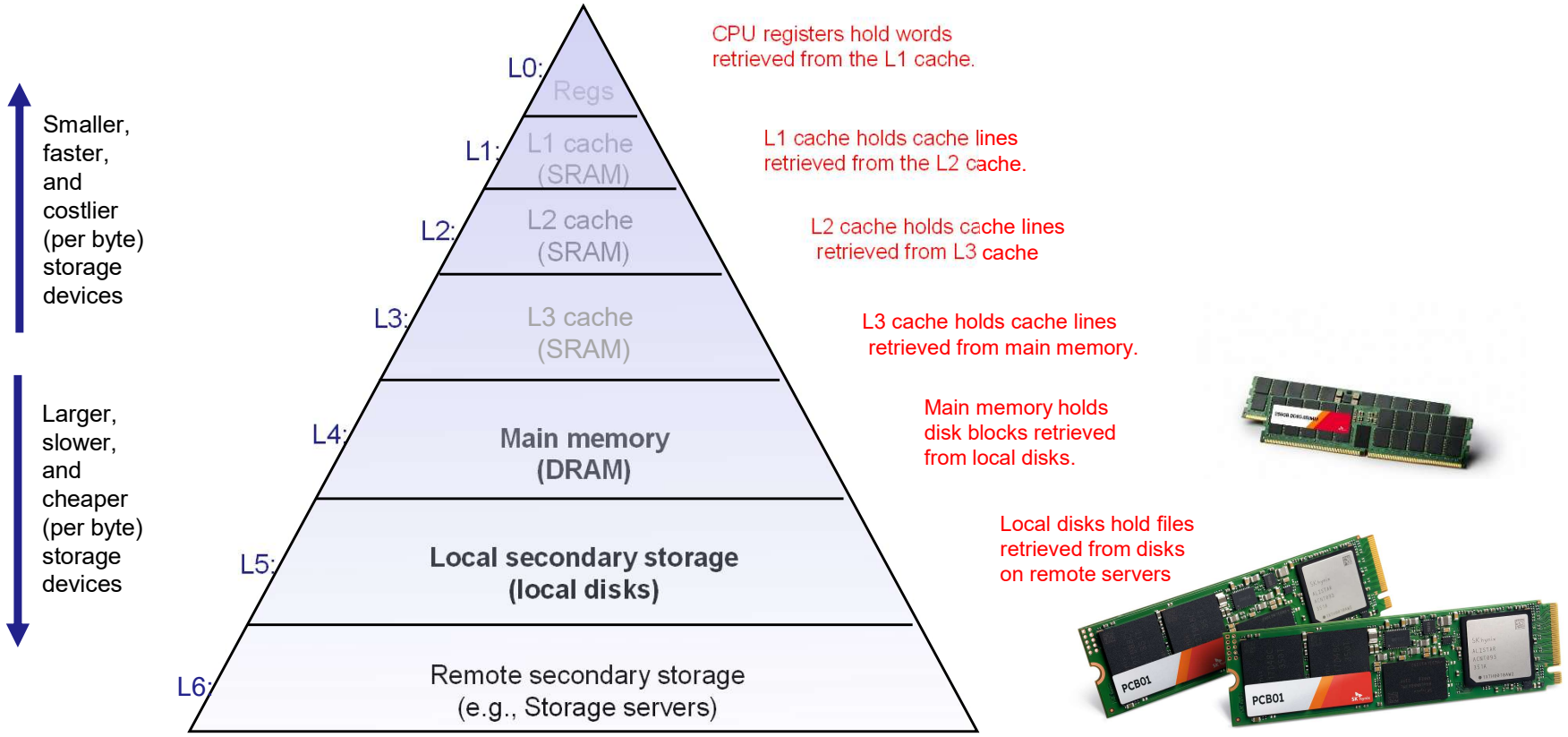
Sogang University

Linux 운영체제 및 응용 (GITF315-01)

R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, 2nd ed. Arpaci-Dusseau Books, 2018. [Online]. Available: https://pages.cs.wisc.edu/~remzi/OSTEP/

Beyond Physical Memory: Mechanisms

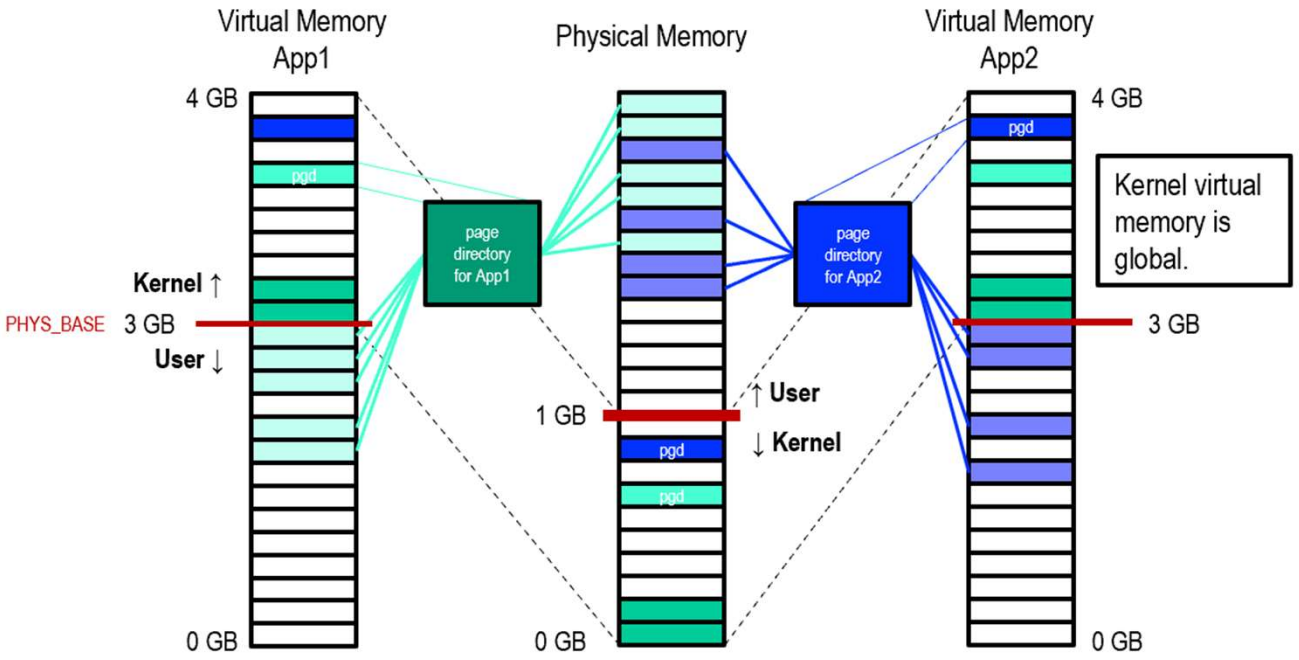
- **Use part of disk as memory.**
  - OS need a place to stash away portions of address space that currently aren't in great demand.



Motivation: Virtual Memory

- **Goal: Run processes even when physical memory is limited**
  - A single process can occupy a very large address space.
  - Multiple processes should be run in the same physical address space.
  - A process should run independent of amount of physical memory correctly.

- **Virtual Memory**
  - Provides the illusion of a larger physical memory

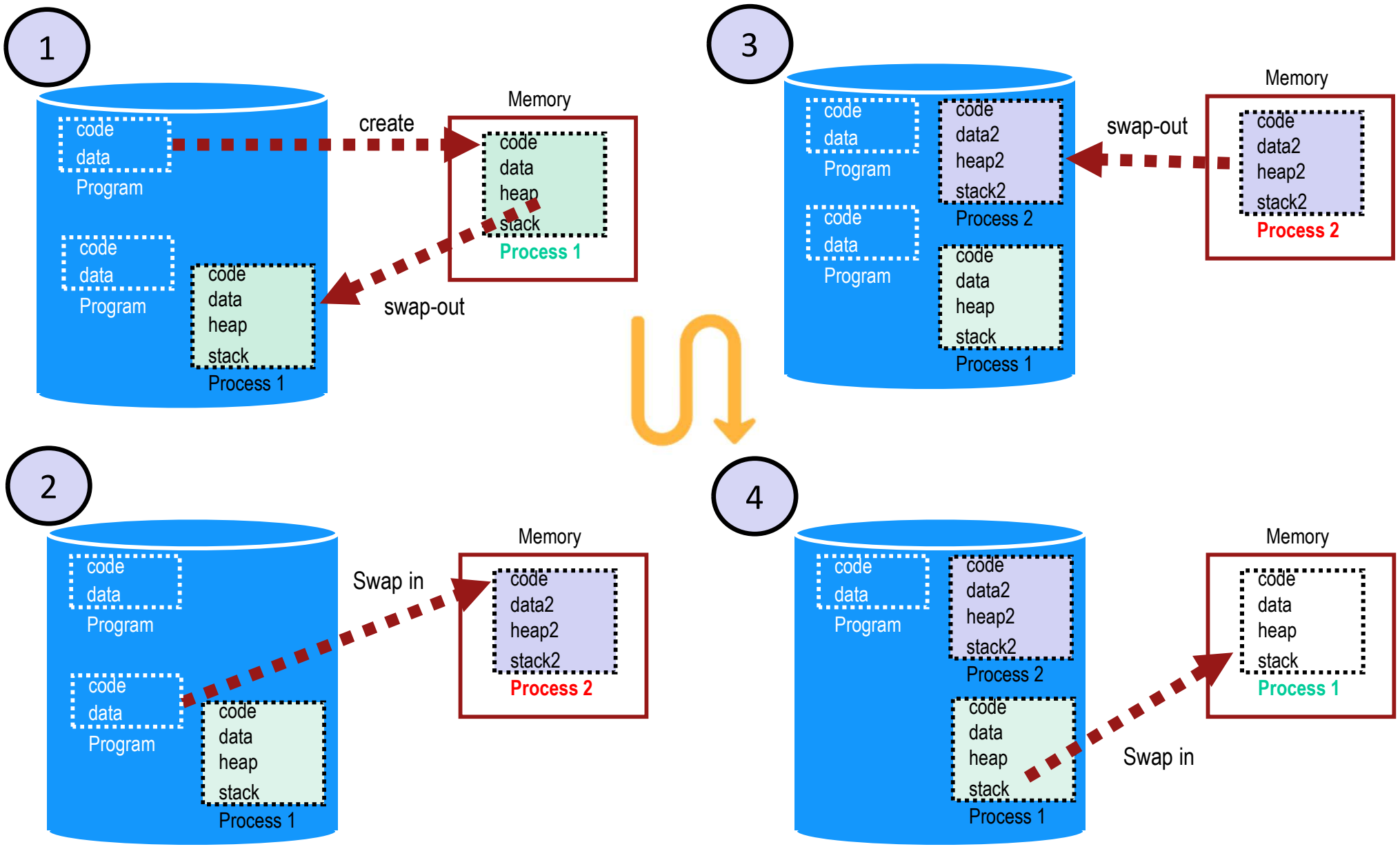


Swapping and Demand Paging

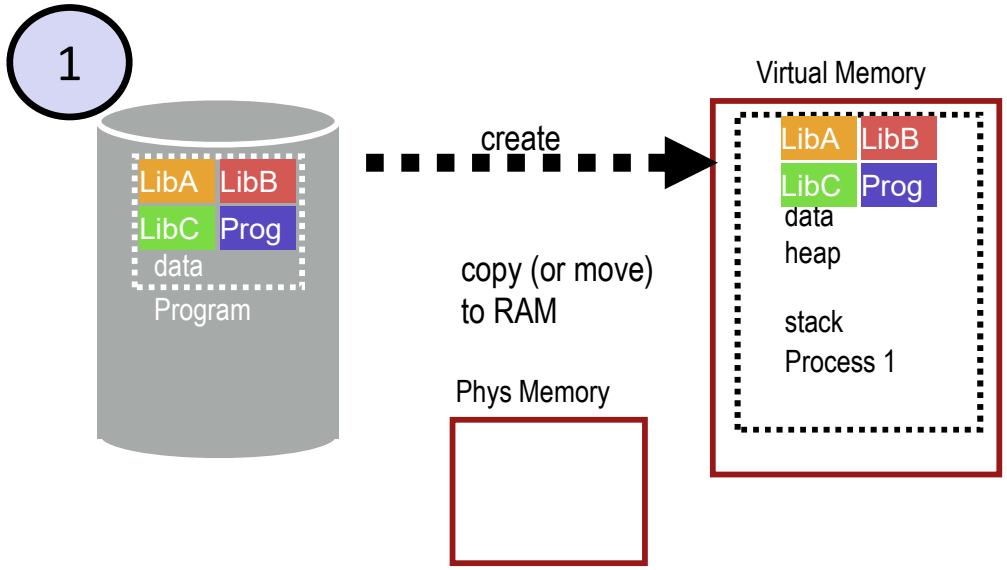
- **Swapping**
  - Swapping enables multiple processes to share limited physical memory.
  - When physical memory is exhausted, the **OS swaps out a victim process from DRAM to disk**, freeing space for another process to run.

- **Demand Paging**
  - Unlike swapping, which moves an entire process in and out of memory, **demand paging performs swapping at page granularity**.
  - Pages are loaded into memory only when they are actually accessed.

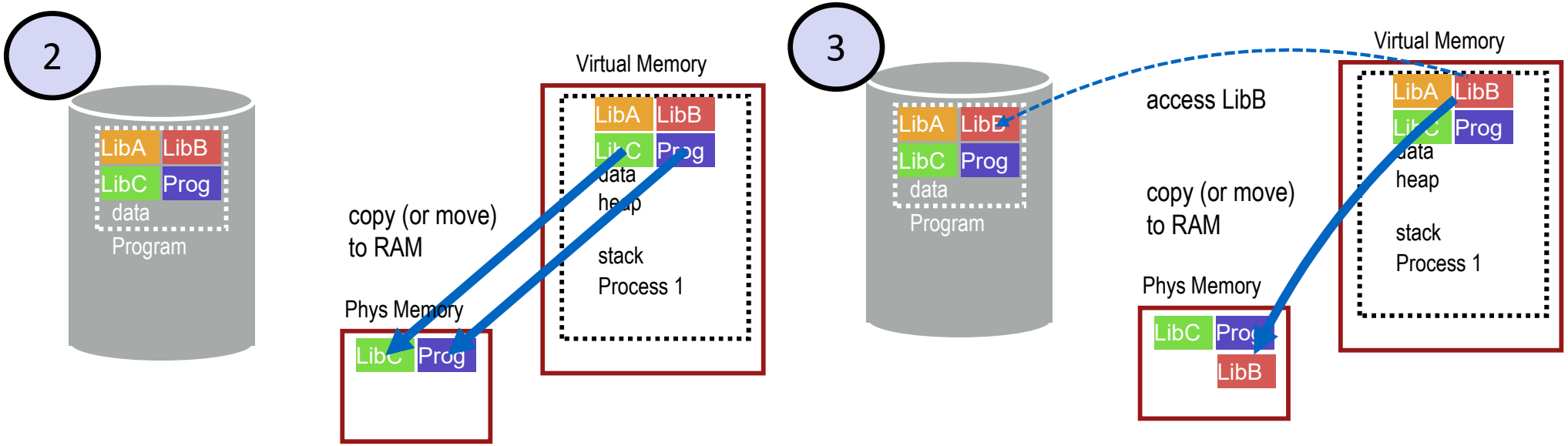
Example: Swapping



Example: Demand Paging



- Many large libraries, some of which are rarely/never used
- How to avoid wasting physical pages to back rarely used virtual pages?



Locality of Reference

- **Leverage locality of reference within processes**
  - **Spatial**: reference memory addresses **near** previously referenced addresses
  - **Temporal**: reference memory addresses that have referenced in the past
  - Processes spend majority of time in small portion of code
    - Estimate: 90% of time in 10% of code

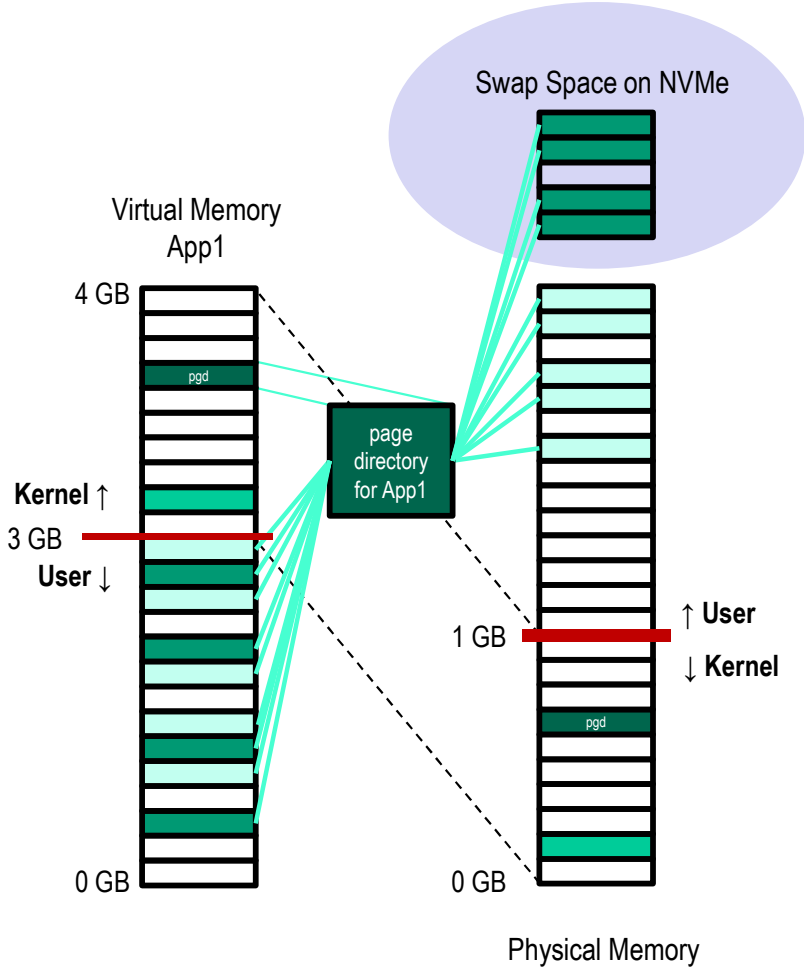
- **Implication:**
  - Process only uses small amount of address space at any moment
  - Only small amount of address space must be resident in physical memory

Virtual Memory

- **Idea for Implementation**
  - The OS keeps **unreferenced pages on disk**.
  - A process can execute even when **not all pages are resident in main memory**.
  - The OS and hardware cooperate to provide the **illusion of a large memory**.
    - The program behaves as if its entire address space were in main memory.
- **Mechanism**
  - The OS must identify **where each page resides**: in memory or on disk.
  - This is supported by the **present bit in the page table**.
- **Policy**
  - The OS must decide **which pages remain in memory** and **which are evicted to disk**.
  - This decision is governed by a **page replacement algorithm**.

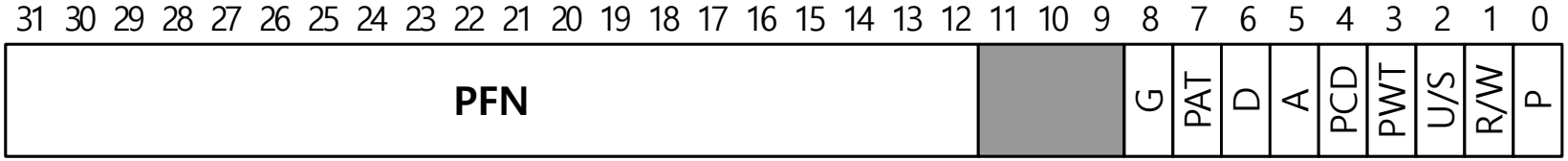
Virtual Memory & Paging/Page Table

- **What Is in the Page Table?**
  - The **page table** is a data structure that maps **virtual addresses** to **physical locations**.
  - The OS indexes the page table using the **Virtual Page Number (VPN)** and retrieves the corresponding **page table entry (PTE)**.
  - Each virtual page is associated with one of the following locations:
    - **DRAM** (resident in physical memory)
    - **Disk** (swapped out)
    - **Nothing** (not allocated / free)



Common Flags of a Page Table Entry (PTE)

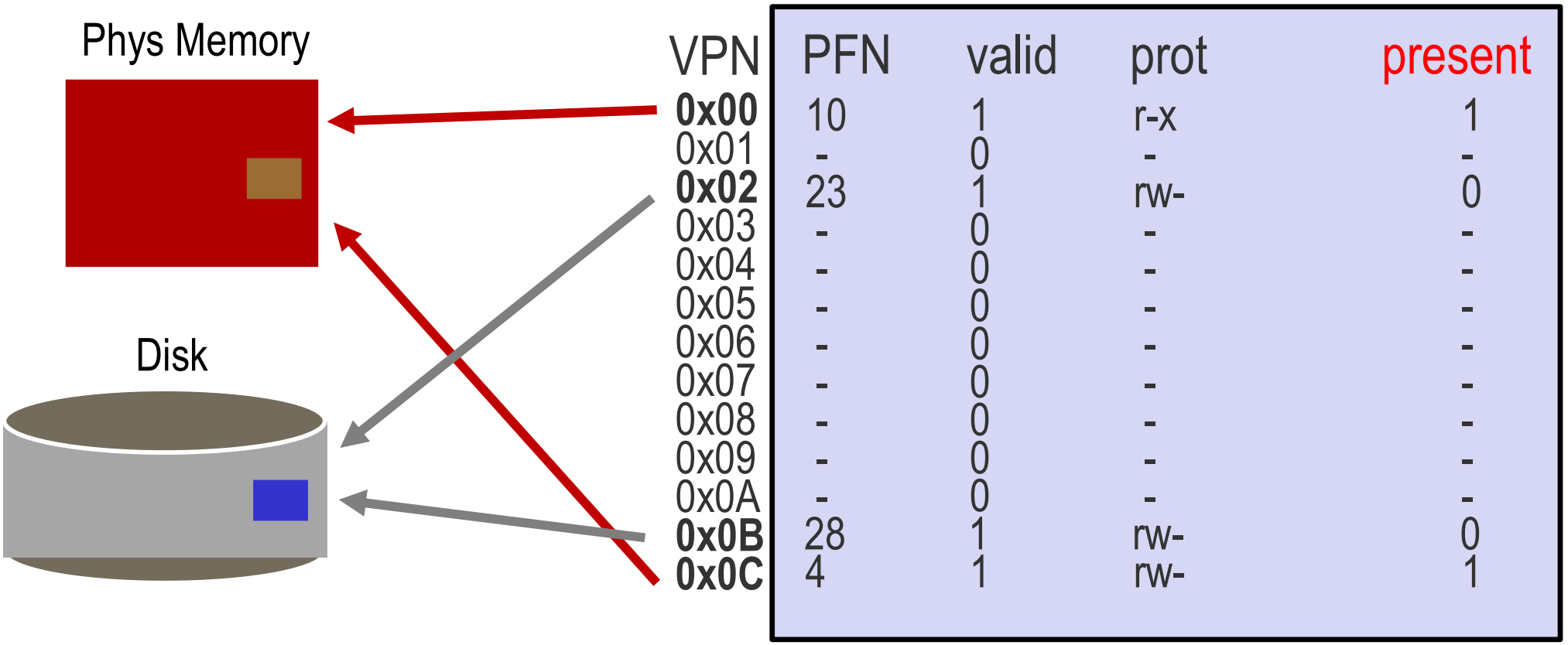
- P: present
- R/W: read/write bit
- U/S: supervisor
- A: accessed bit
- D: dirty bit
- PFN: the page frame number



An x86 Page Table Entry(PTE)

- **Present Bit: Indicates whether the page is currently in physical memory or has been swapped out to disk.**
- **Protection Bits**: Specify whether the page can be read, written, or executed.
- **Supervisor Bit**: Indicates whether it is accessable by the kernel or user.
- **Dirty Bit**: Indicates whether the page has been modified since it was loaded into memory.
- **Accessed Bit**(Reference Bit): Indicates whether the page has been accessed recently.

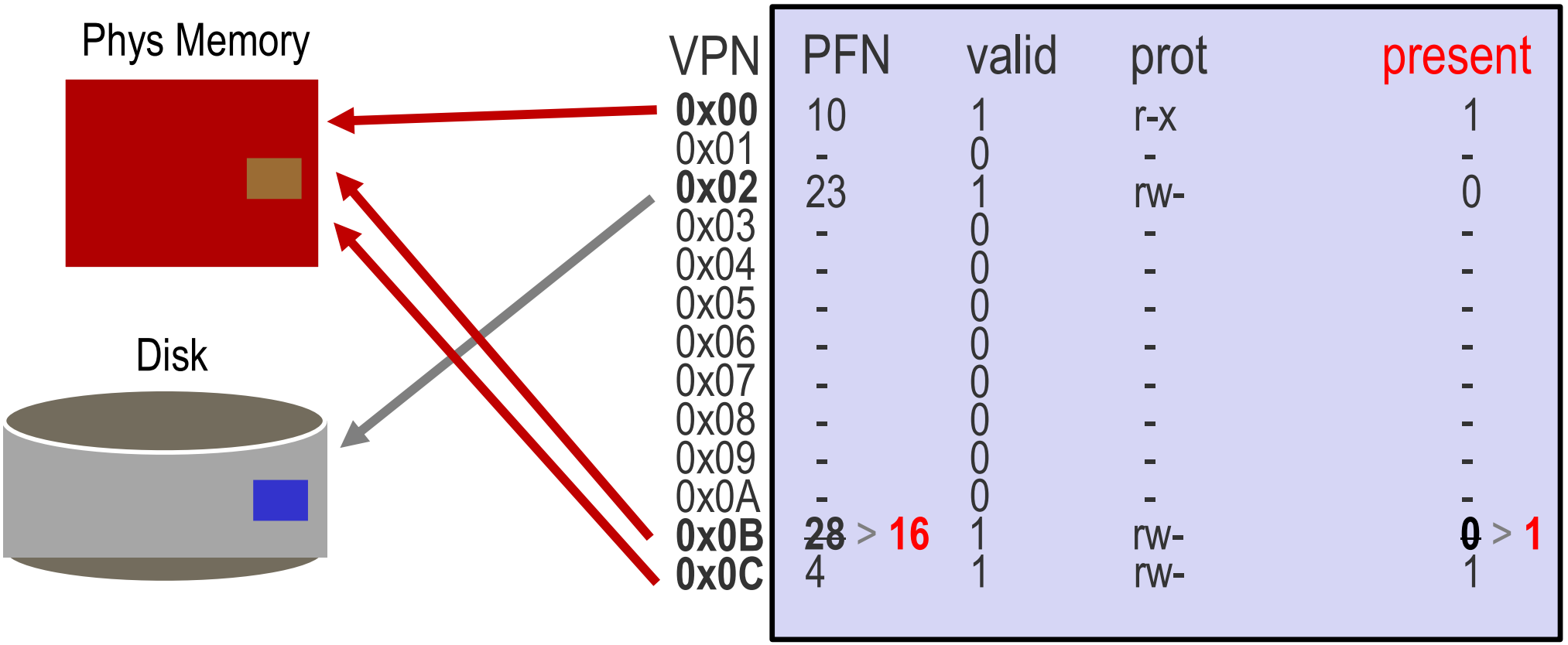
Present Bit in Page Table



Page Table

What if access vpn 0x0B?

Present Bit in Page Table

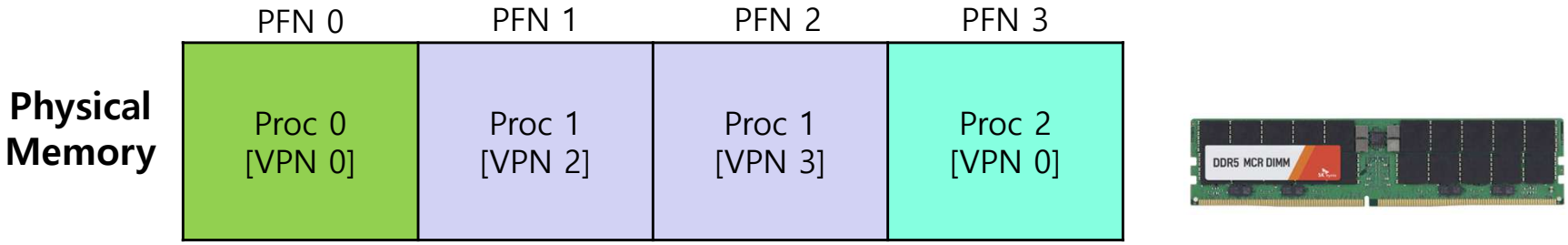


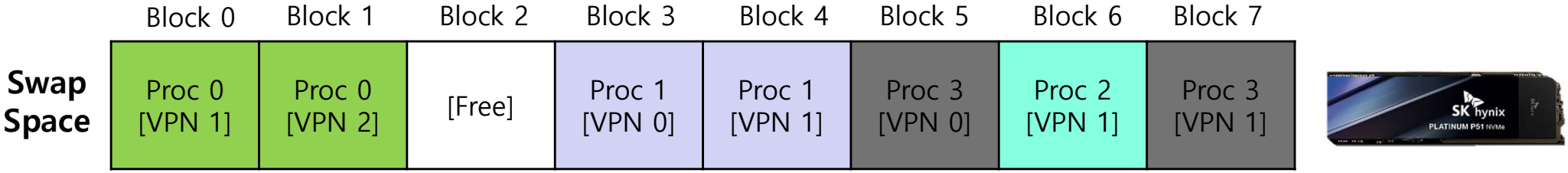
Page Table

What if access vpn 0x0B?

Swap Space

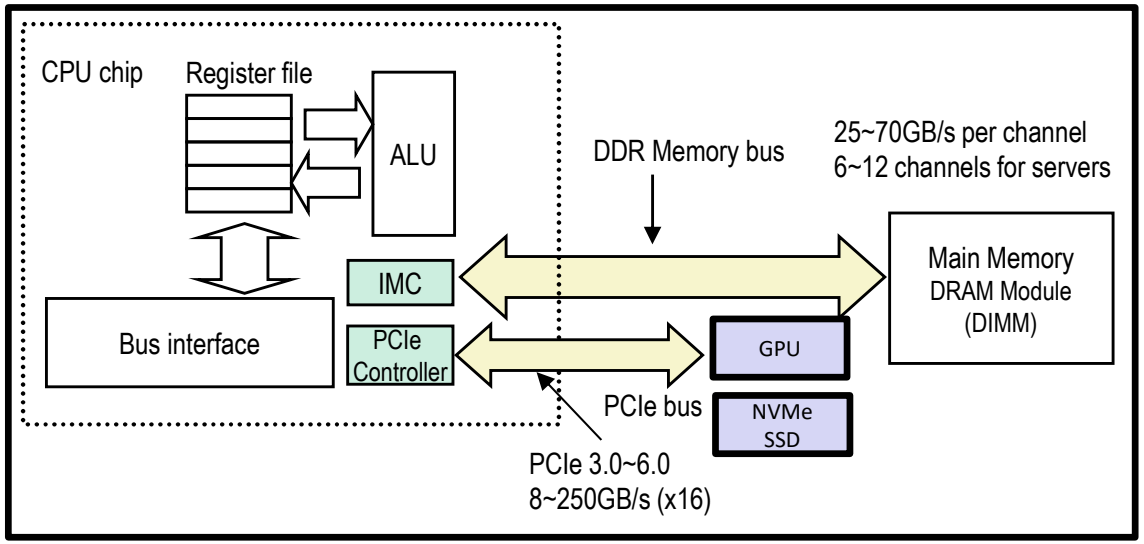
- Reserve some space on the disk for moving pages back and forth.
- OS needs to remember the swap space, in page-sized unit.

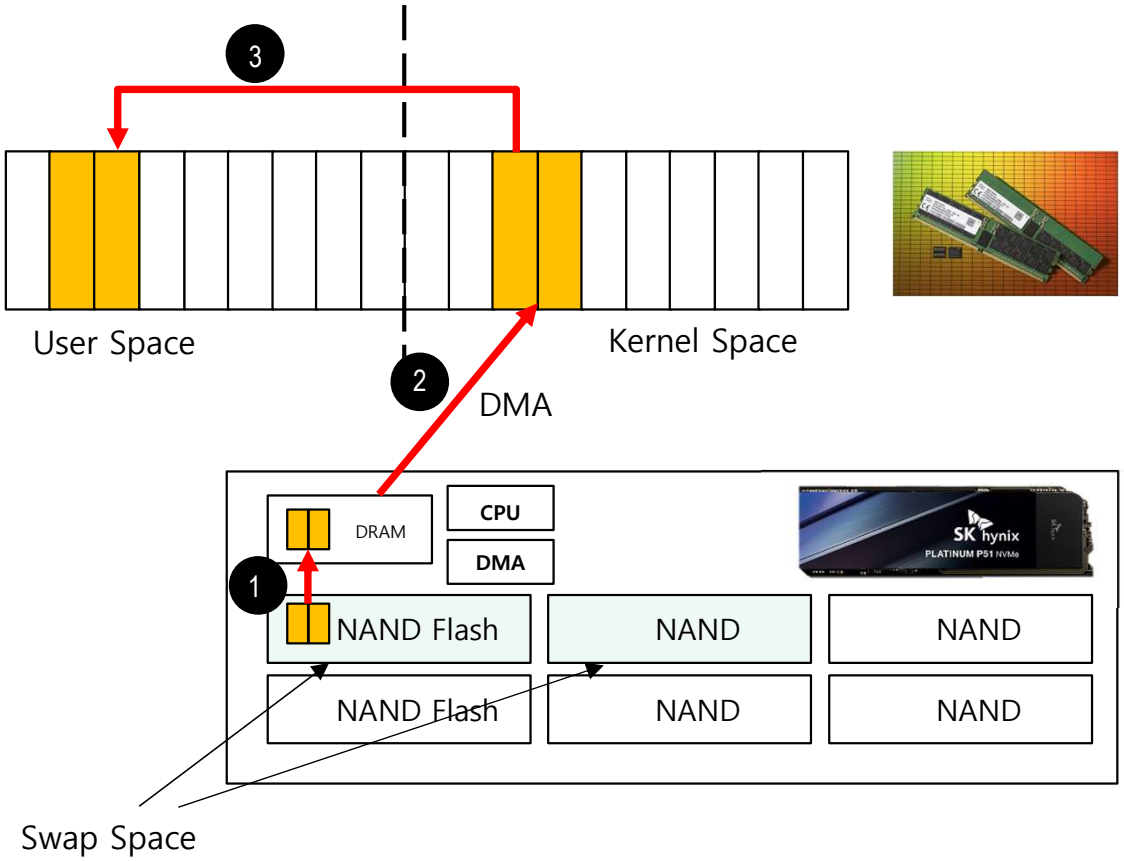


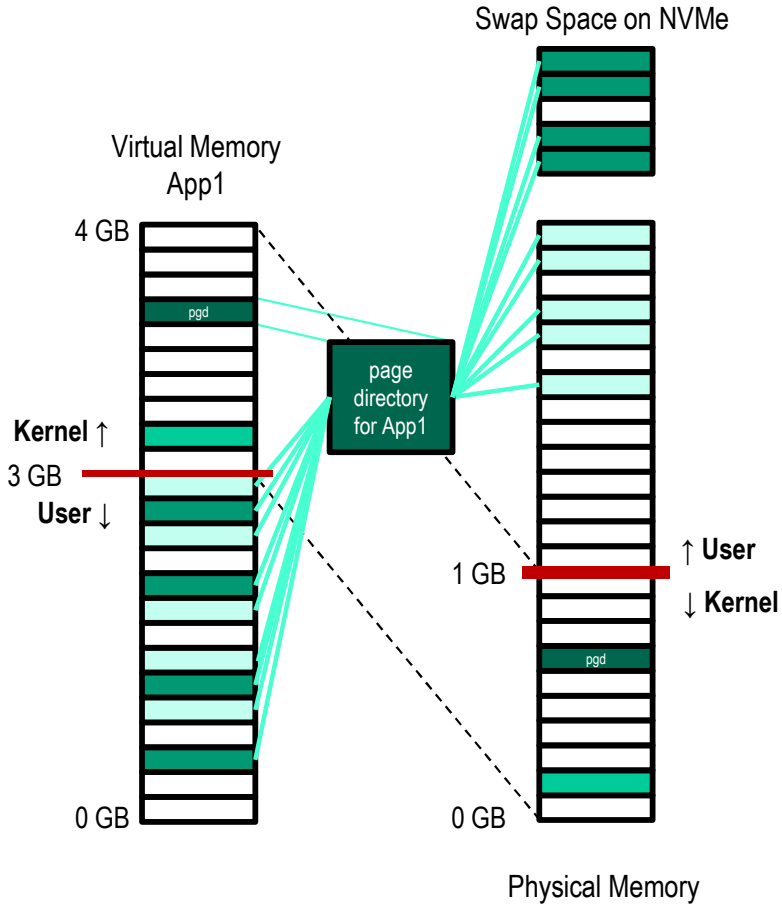


Swap Space

- **Swapping**







Page Fault Control Flow

① Memory Reference

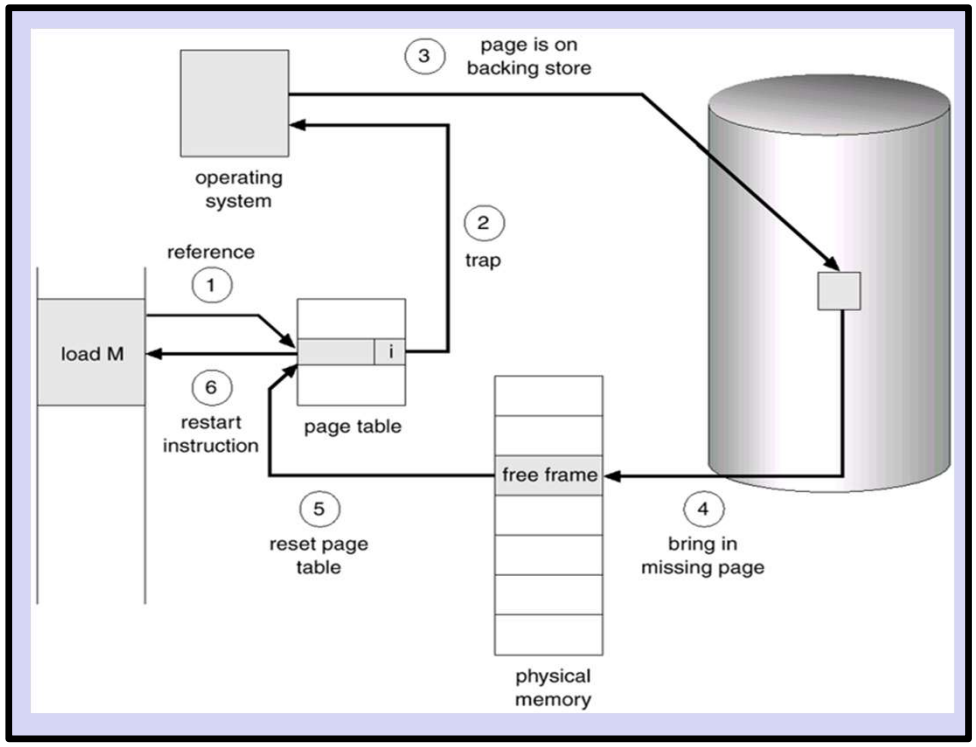
- The CPU issues a memory access using a virtual address.
- The page table is consulted to translate the address.

② Page Fault Trap

- If the Present bit is 0, the MMU raises a page fault exception.
- Control is transferred to the operating system.

③ Page Validation

- The OS checks whether the referenced page is a valid page (e.g., allocated to the process).
- If the page is invalid, the OS terminates the process with a segmentation fault.



Page Fault Control Flow

④ Page Retrieval

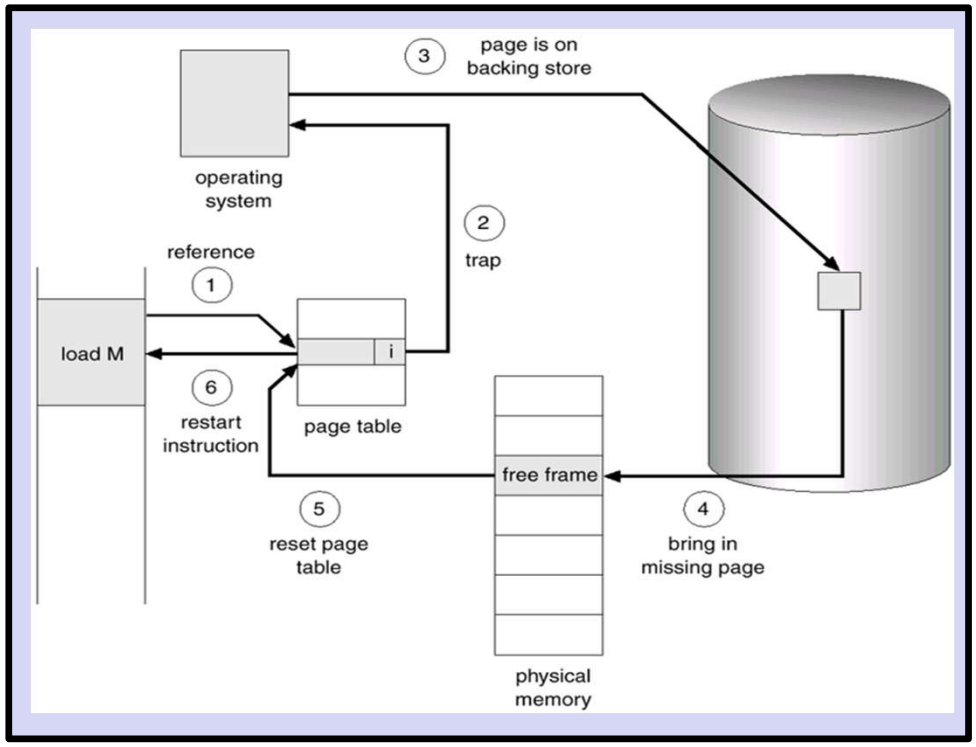
- If the page is valid but not resident, the OS locates the page on disk.
- The page is read from disk into a free page frame in physical memory.

⑤ Page Table Update

- The OS updates the page table entry with the new physical frame number.
- The Present bit is set to 1.

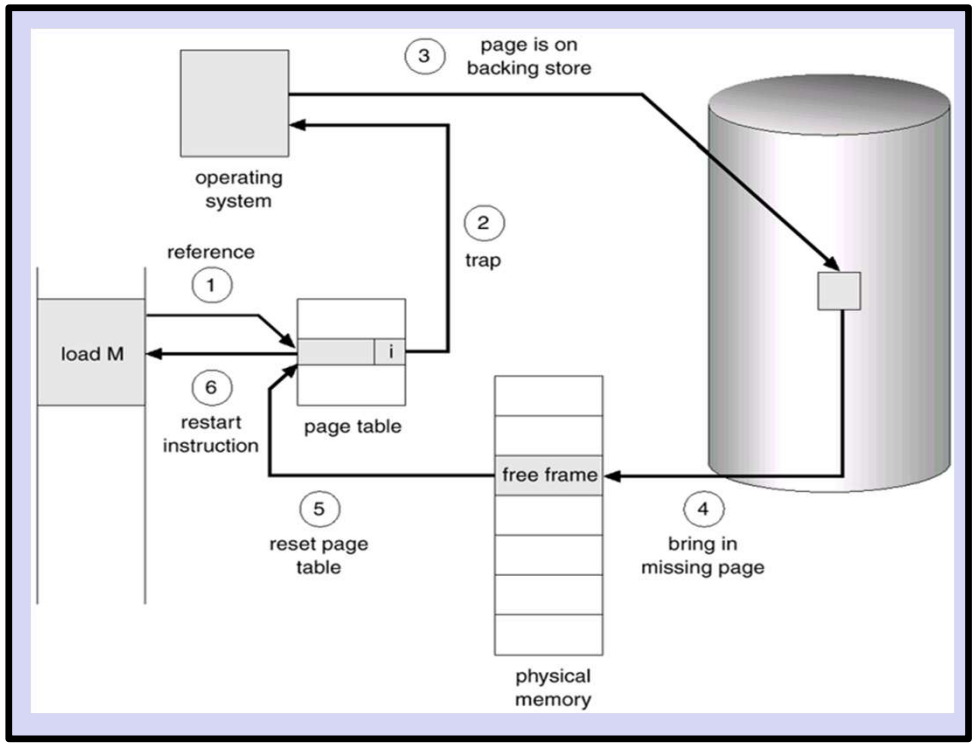
⑥ Instruction Restart

- The faulting instruction is restarted.
- Execution continues as if the page fault had never occurred.



Page Fault Control Flow

- **Page Replacement**
  - If no free page frames are available, the OS must **evict an existing page** to make room for the incoming page.
  - The selection of a victim page is determined by the **page replacement policy** (e.g., FIFO, LRU, Clock).
  - If the **evicted page** is **dirty**, it must be **written back to disk** before reuse.



Page Fault Control Flow – Hardware

```
1:      VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2:      (Success, TlbEntry) = TLB_Lookup(VPN)
3:      if (Success == True) // TLB Hit
4:          if (CanAccess(TlbEntry.ProtectBits) == True)
5:              Offset = VirtualAddress & OFFSET_MASK
6:              PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7:              Register = AccessMemory(PhysAddr)
8:          else RaiseException(PROTECTION_FAULT)
9:      else // TLB Miss
10:         PTEAddr = PTBR + (VPN * sizeof(PTE))
11:         PTE = AccessMemory(PTEAddr)
12:         if (PTE.Valid == False)
13:             RaiseException(SEGMENTATION_FAULT)
14:         else
15:             if (CanAccess(PTE.ProtectBits) == False)
16:                 RaiseException(PROTECTION_FAULT)
17:             else if (PTE.Present == True)
18:                 // assuming hardware-managed TLB
19:                 TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
20:                 RetryInstruction()
21:             else if (PTE.Present == False)
22:                 RaiseException(PAGE_FAULT)
```

<Hardware-managed TLB>

Page Fault Control Flow – Software

```
1:        PFN = FindFreePhysicalPage()
2:        if (PFN == -1) // no free page found
3:                PFN = EvictPage() // run replacement algorithm
4:        DiskRead(PTE.DiskAddr, pfn) // sleep (waiting for I/O)
5:        PTE.present = True // update page table with present
6:        PTE.PFN = PFN // bit and translation (PFN)
7:        RetryInstruction() // retry instruction
```

- The OS must find a physical frame for the soon-be-faulted-in page to reside within.
- If there is no such page, waiting for the replacement algorithm to run and kick some pages out of memory.

Page Replacement

- **`PTE.Present == False`** // **page fault**

  ① Trap into OS (not handled by hardware)

  ② OS selects victim page in memory to replace
  - Write victim page out to disk if modified (add `dirty` bit to PTE)

  ③ OS reads referenced page from disk into memory

  ④ Page table is updated, **`present`** bit is set

  ⑤ Process continues execution

Performance Analysis of Demand Paging

- **Performance of Demand Paging**
  - **Demand paging** can have a **significant impact on overall system performance**, since page faults are several orders of magnitude slower than normal memory accesses.

- **Page Fault Rate**
  - Let **p** denote the **page fault rate**, where
$$0 \leq p \leq 1$$
  - If **p = 0**, there are **no page faults** (all references hit in memory).
  - If **p = 1**, **every memory reference causes a page fault**.

Performance Analysis of Demand Paging

- **Effective Access Time (EAT)**
  - The **Effective Access Time (EAT)** represents the average time to access a memory location, accounting for both normal accesses and page faults.

$$\text{EAT} = (1 - p) \times \textit{Memory Access Time} \; + \; p \times (\textit{Page Fault Overhead} + [\textit{Swap Page Out}] + \textit{Swap Page In} + \textit{Instruction Restart Overhead})$$

  - The **swap page out cost** is incurred **only if the victim page is dirty**.
  - Page fault handling involves **disk I/O**, making even a small page fault rate highly expensive.

Demand Paging Example

- **Example 1:**
  - **Memory access time = 1 microsecond (usec)**
  - 50% of the pages that are being replaced have been modified and therefore need to be swapped out.
  - Swap Page Time = 10 msec = 10,000 usec
  - Ignore ”restart overhead”
  - EAT = $(1 - p) \times 1 + p \times (15000) \sim 1 + 15000\,p$ (in usec)

Assumption: When an easy error occurs, the process that causes the page fault does not continue until the page fault has been processed. CPU is idle until page fault handling is complete. That is, no other process is scheduled to occupy the CPU.

Demand Paging Example

- **Example 2:**
  - **Memory access time = 200 nsec**
  - Average page-fault service time = 8 msec
    - The page-fault service time contains swap page in and out time and restart overhead.
  - EAT = (1 – p) x 200 + p (8 msec)

    = (1 – p) x 200 + p x 8,000,000 = 200 + p x 7,999,800
  - If one access out of 1,000 causes a page fault, then EAT = 8.2 usec
  - This is a slowdown by a factor of 40 !!

Assumption: When an easy error occurs, the process that causes the page fault does not continue until the page fault has been processed. CPU is idle until page fault handling is complete. That is, no other process is scheduled to occupy the CPU.

Virtual Memory Policies

- **Goal: Minimize number of page faults**
  - Page faults require milliseconds to handle (reading from disk)
  - Implication: Plenty of time for OS to make good decision

- **OS has two decisions**
  1) **Page selection**
     - When should a page (or pages) on disk be brought into memory?
  2) **Page replacement**
     - Which resident page (or pages) in memory should be thrown out to disk?

1) Page Selection

- **When should a page be brought from disk into memory?**

- **Demand paging: Load page only when page fault occurs**
  - Intuition: Wait until page must absolutely be in memory
  - When process starts: No pages are loaded in memory
  - Problems: Pay cost of page fault for every newly accessed page
- **Prepaging (anticipatory, prefetching): Load page before referenced**
  - OS predicts future accesses (oracle) and brings pages into memory early
  - Works well for some access patterns (e.g., sequential)
  - Problems?
- **Hints: Combine above with user-supplied hints about page references**
  - User specifies: may need page in future, don't need this page anymore, or sequential access pattern, ...
  - Example: `madvise()` in Unix

2) Page Replacement

- **In case of no free frame -> page replacement**

- **Page replacement – find some page in memory, but not really in use, swap it out**
  - Need an algorithm which will result in minimum number of page faults (Same page may be brought into memory several times)

- **Which page in main memory should selected as victim?**
  - Write out victim page to disk if modified (dirty bit set)
  - If victim page is not modified (clean), just discard

When to Perform Replacement

1) Lazy approach

- In a **lazy approach**, the operating system waits until **physical memory** is **completely full**, and only then selects a page to evict in order to make room for a new page.
- This approach is **unrealistic in practice**, as it can lead to excessive page faults and poor system responsiveness.

When to Perform Replacement

2) Swap Daemon / Page Daemon

- Modern operating systems employ a **background kernel thread**, often referred to as the **swap daemon** or **page daemon**, to proactively manage memory pressure.
- When the number of free pages falls **below the Low Watermark (LW)**, the page daemon is activated.
- The daemon **evicts pages in the background** until the number of free pages reaches the **High Watermark (HW)**.
- This proactive reclamation helps ensure that free pages are available when page faults occur, thereby reducing fault latency.

Key Insight

- **Page replacement** is typically **performed proactively** in the **background**, rather than reactively at the moment memory becomes exhausted.

Page Replacement Algorithm
(or Swapping Policy)

Goal of Cache Management

- **Minimize the number of cache misses.**
- **Minimize the Average Memory Access Time (AMAT)**

$$AMAT = (P_{Hit} * T_M) + (P_{Miss} * T_D)$$

Arguement	Meaning
T_M	The cost of accessing memory
T_D	The cost of accessing disk
P_{Hit}	The probability of finding the data item in the cache(a hit)
P_{Miss}	The probability of not finding the data in the cache(a miss)

Page Replacement Policies

1) Optimal Replacement Policy
2) FIFO (First-In and First-Out) Policy
3) Random Policy
4) LRU (Least-Recently Used) Policy
5) LFU (Least-Frequently Used) Policy
6) …

No single policy is optimal for all workloads…

- Practical systems use approximations of LRU (e.g., Clock, Second-Chance)

The Optimal Replacement Policy

- **The Optimal Page Replacement Policy**
  - The **optimal replacement policy** results in the **minimum possible number of page faults (misses)** over the entire execution.
  - **Replacement rule**
    - Always evict the page that will be **accessed farthest in the future**.
    - This guarantees the **fewest possible cache/page misses** for any given reference string.
- **Practical Significance**
  - The optimal policy **cannot be implemented in practice**, because it requires **perfect knowledge of future memory accesses**.
  - Therefore, it is used **only as a theoretical benchmark**.
- **Key Insight**
  - **Optimal replacement serves as a comparison point**, allowing us to evaluate how close practical algorithms (e.g., LRU, Clock) come to the theoretical optimum.

Tracing the Optimal Policy

Reference Row

0 1 2 0 1 3 0 3 1 2 1

Hit rate is $\frac{Hits}{Hits+Misses} = \mathbf{54.6\%}$

Access	Hit/Miss?	Evict	Resulting Cache State
0	Miss		0
1	Miss		0,1
2	Miss		0,1,2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	2	0,1,3
0	Hit		0,1,3
3	Hit		0,1,3
1	Hit		0,1,3
2	Miss	3	0,1,2
1	Hit		0,1,2

A Simple Policy: FIFO

- **A Simple Replacement Policy: FIFO**
  - Pages are **inserted into a queue** when they first enter physical memory.
  - When a page replacement is required, the page at the **front of the queue**—that is, the page that **entered memory first (First-In)**—is selected for eviction.
- **Characteristics**
  - **Simple to implement**, with low bookkeeping overhead.
  - Does **not consider page usage or importance**.
  - May evict frequently used pages, leading to **poor performance** in some workloads.
- **Key Insight**
  - FIFO replacement is easy to implement but oblivious to access patterns, which can result in unnecessary page faults.

Tracing the FIFO Policy

Reference Row

0 1 2 0 1 3 0 3 1 2 1

Access	Hit/Miss?	Evict	Resulting Cache State
0	Miss		0
1	Miss		0,1
2	Miss		0,1,2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	1,2,3
0	Miss	1	2,3,0
3	Hit		2,3,0
1	Miss		3,0,1
2	Miss	3	0,1,2
1	Hit		0,1,2

Hit rate is $\frac{Hits}{Hits+Misses} = \mathbf{36.4\%}$

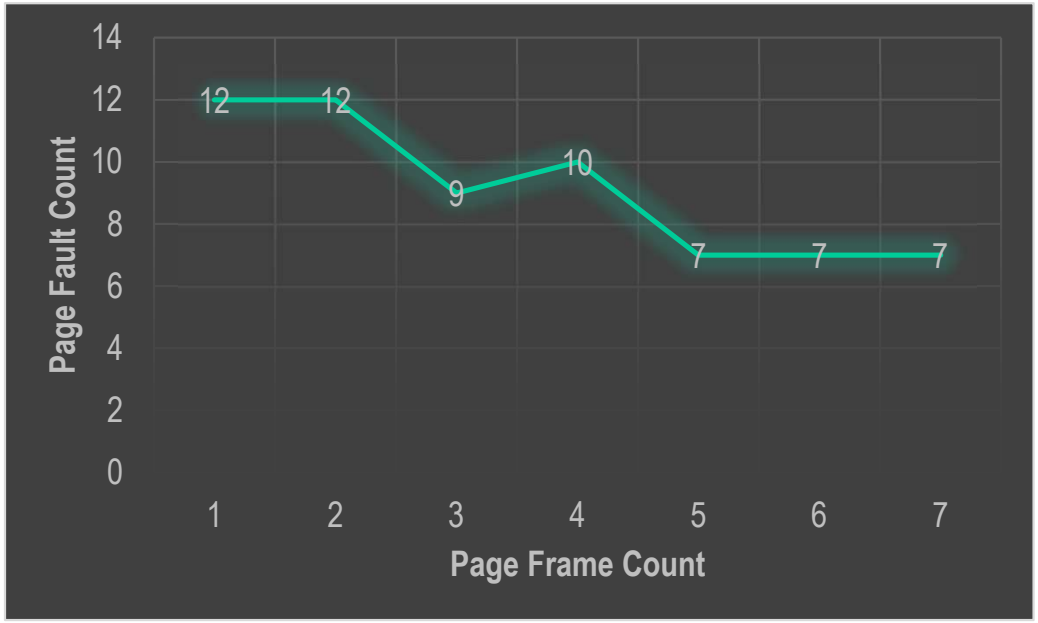
- **Optimal Policy**

Hit rate is $\frac{Hits}{Hits+Misses} = \mathbf{54.6\%}$

Belady's Anomaly

- **Belady's Anomaly**
  - In general, we expect the **cache hit rate to increase** (and the miss rate to decrease) as the **cache size grows**.
  - However, under the **FIFO (First-In, First-Out) replacement policy**, the **opposite behavior can occur**:
    - **Increasing the cache size may lead to a higher miss rate**.
  - This **counterintuitive phenomenon** is known as **Belady's Anomaly**.

Reference Row
0 1 2 0 1 3 0 3 1 2 1



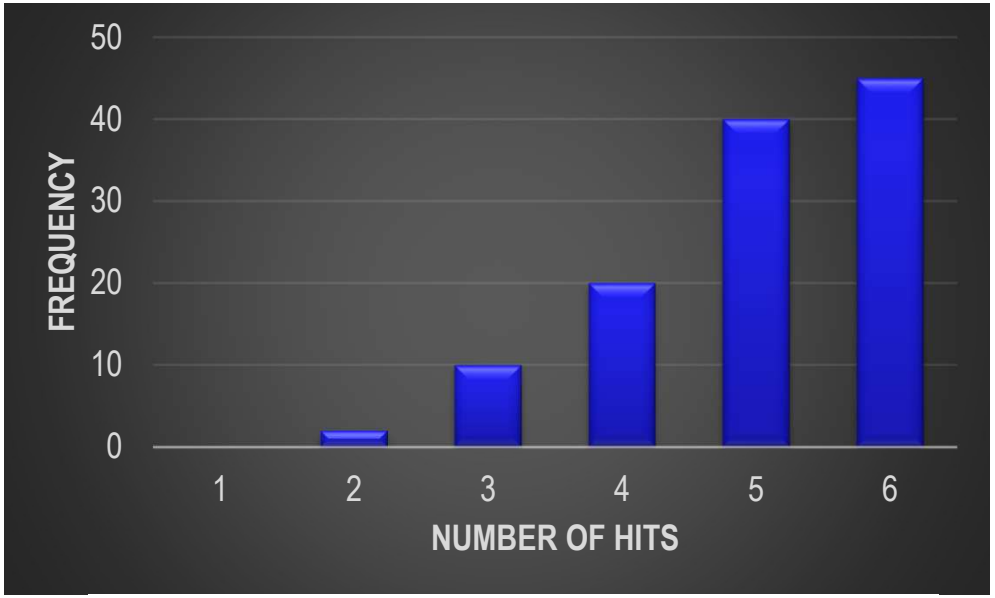
Belady's Anomaly

- **Explanation**
  - FIFO evicts pages solely based on **arrival order**, without considering how frequently or recently a page is accessed.
  - As a result, adding more page frames can change the eviction order in a way that **removes useful pages earlier**, increasing the number of page faults.
- **Key Insight**
  - **More memory does not always mean better performance under FIFO**. Replacement policies that do not preserve inclusion properties can exhibit anomalous behavior.
- **Important Note**
  - Belady's Anomaly does not occur in stack algorithms such as:
    - Optimal (OPT)
    - Least Recently Used (LRU)

Another Simple Policy: Random

- **Picks a random page to replace under memory pressure.**
  - It doesn't really try to be too intelligent in picking which blocks to evict.
  - Random does depends entirely upon how lucky Random gets in its choice.
- **Sometimes, Random is as good as optimal, achieving 6 hits on the example trace.**

Access	Hit/Miss?	Evict	Resulting Cache State
0	Miss		0
1	Miss		0,1
2	Miss		0,1,2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	1,2,3
0	Miss	1	2,3,0
3	Hit		2,3,0
1	Miss	3	2,0,1
2	Hit		2,0,1
1	Hit		2,0,1



Random Performance over 10,000 Trials

Using History

- **Learn from past behavior and exploit access history.**
  - There are two types of historical information commonly used in page replacement policies.

Historical Information	Meaning	Algorithms
recency	The more recently a page has been accessed, the more likely it will be accessed again	LRU
frequency	If a page has been accessed many times, It should not be replcaed as it clearly has some value	LFU

- **Key Insight**
  - **Past access patterns provide useful predictors of future behavior**, enabling smarter replacement decisions than simple policies like FIFO.

Using History : LRU

Reference Row

0 1 2 0 1 3 0 3 1 2 1

Access	Hit/Miss?	Evict	Resulting Cache State
0	Miss		0
1	Miss		0,1
2	Miss		0,1,2
0	Hit		1,2,0
1	Hit		2,0,1
3	Miss	2	0,1,3
0	Hit		1,3,0
3	Hit		1,0,3
1	Hit		0,3,1
2	Miss	0	3,1,2
1	Hit		3,2,1

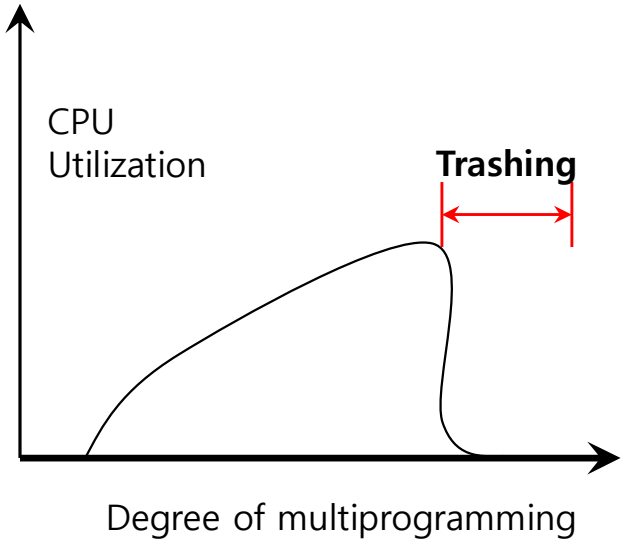
Hit rate is $\frac{Hits}{Hits+Misses} = \mathbf{54.6\%}$

- **Optimal Policy**

Hit rate is $\frac{Hits}{Hits+Misses} = \mathbf{54.6\%}$

Thrashing

- **Thrashing**
  - Thrashing occurs when memory is oversubscribed and the combined working sets of running processes exceed the available physical memory.
    - The system spends most of its time **paging** rather than executing useful work.



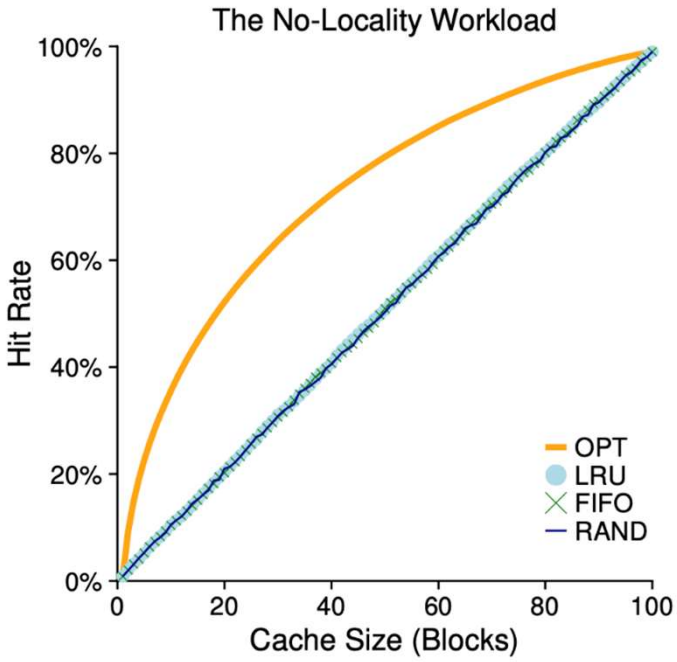
- **Consequences**
  - High page fault rate, Low CPU utilization
  - Performance degrades as the system becomes dominated by memory I/O
- **System Response**
  - Reduce the degree of multiprogramming:
    - Temporarily suspend or swap out a subset of processes.
    - Allow the **working sets of the remaining processes** to fit in memory.

Virtual Memory & Swapping: Summary

- **Virtual Memory** enables programs to run with an address space larger than physical memory → Illusion of large, contiguous memory
- **Swapping** moves inactive memory pages between RAM and secondary storage (disk/SSD) → Frees physical memory for active working sets
- **Page Fault** triggers swapping
  - Required page is fetched from disk into memory
  - Victim page may be evicted based on a replacement policy
- **Page Replacement Policies** determine which page to evict
  - Optimal (theoretical baseline)
  - LRU (temporal locality)
- Performance Impact
  - Disk I/O is orders of magnitude slower than memory
  - Excessive swapping leads to thrashing and severe slowdown
- Swapping is essential for memory virtualization, but efficient page replacement and locality-aware execution are critical for performance.

Workload Example : The No-Locality Workload

- **Each memory reference accesses a random page from the set of accessed pages.**
  - The workload accesses 100 unique pages over time.
  - Each reference selects the next page uniformly at random from this set.



Key Observations

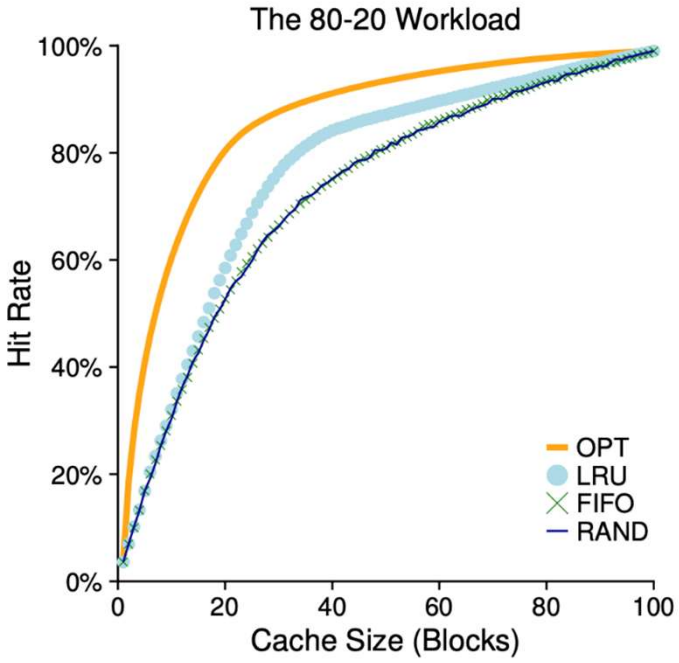
- This workload exhibits **no temporal or spatial locality**.
- As a result, **replacement policies such as LRU, FIFO, and RAND perform similarly**.
- The **OPT policy** still provides an upper bound, but its advantage is limited.

Important Insight

- When the cache is large enough to hold the entire working set, the choice of replacement policy no longer matters.

Workload Example : The 80-20 Workload

- **This workload exhibits locality: 80% of memory references are made to 20% of the pages.**
  - The remaining 20% of references are made to the remaining 80% of the pages.



Key Observations

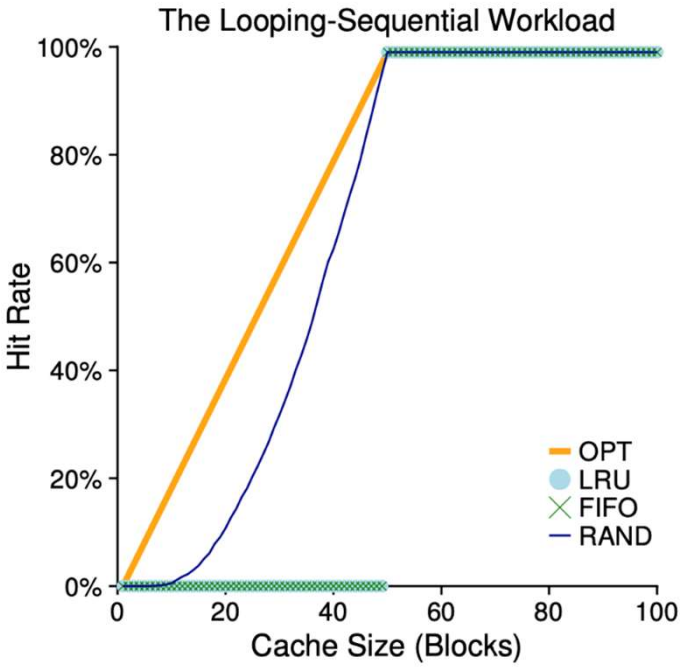
- This workload demonstrates **strong temporal locality**.
- **LRU significantly outperforms FIFO and RAND**, especially when the cache size is limited.
- **OPT** provides an upper bound on achievable performance.

Important Insight

- LRU is more effective at retaining frequently accessed ("hot") pages, leading to a higher hit rate.

Workload Example : The Looping Sequential

- **The workload accesses 50 pages sequentially.**
  - Starting from page 0, then 1, … up to page 49.
  - The sequence then loops back to the beginning and repeats.
  - In total, the workload performs 10,000 accesses to 50 unique pages.



Key Observations

- This workload exhibits **strong spatial locality** and **periodic access patterns**.
- When the cache size is **smaller than the working set (50 pages)**:
  - FIFO and LRU suffer **from systematic evictions**, resulting in a low hit rate.
- Once the cache can hold **all 50 pages**:
  - **All policies achieve a near-100% hit rate.**
  - The performance difference between replacement policies disappears.

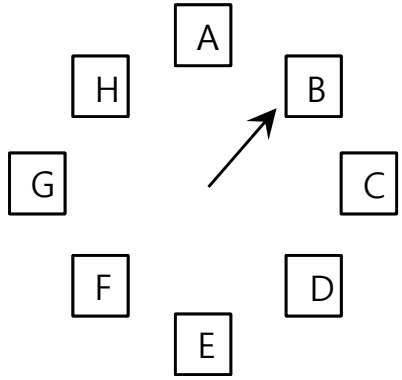
Important Insight

- For looping sequential workloads, cache performance improves dramatically once the cache size reaches the working set size.

Approximating LRU: Clock Algorithm

- **Hardware Support: Use Bit (Reference Bit)**
  - Requires hardware support in the form of a *use bit*.
    - Whenever a page is referenced, the hardware sets the use bit to 1.
    - The hardware never clears the use bit; clearing it is the responsibility of the operating system.

- **The Clock Algorithm**
  - All pages in memory are **organized in a circular list**.
  - A **clock hand** initially points to an arbitrary page.
  - When a page replacement is needed, the algorithm:
    - Examines the page pointed to by the clock hand.
    - **If the use bit is 0**, the page is selected for eviction.
    - **If the use bit is 1**, the OS clears the bit and advances the clock hand.
  - This process continues until a page with use bit = 0 is found.



Use bit	Meaning
0	Evict the page
1	Clear **Use bit** and advance hand

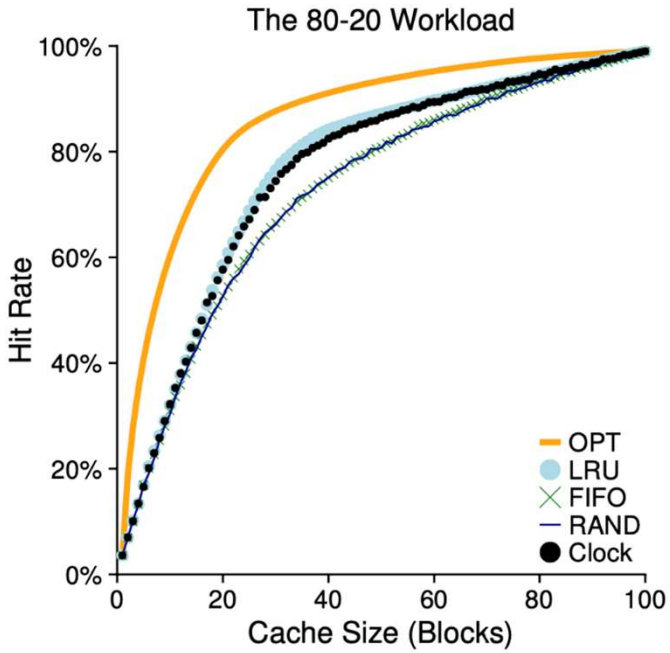
The Clock page replacement algorithm

Important Insight

- The Clock algorithm approximates LRU by giving recently accessed pages a "second chance."

Workload with Clock Algorithm

- **The Clock algorithm does not perform as well as perfect LRU, but it performs significantly better than policies that ignore access history altogether.**



Key Observations

- **OPT** provides the upper bound on achievable hit rate.
- **LRU** closely tracks OPT due to its accurate use of recency information.
- **Clock**:
  - Approximates LRU by retaining recently accessed pages.
  - Performs slightly worse than true LRU due to its coarse-grained history tracking.
- **FIFO** and **RAND**:
  - Do not effectively capture locality.
  - Show consistently lower hit rates under skewed workloads.

Important Insight

- Clock strikes a balance between performance and implementation cost, achieving near-LRU behavior with much lower overhead.

Considering Dirty Pages

- **Modified Bit (Dirty Bit)**
  - The hardware maintains a *modified bit* (also known as the *dirty bit*).
    - If a page **has been modified**, it is marked **dirty** and **must be written back to disk before eviction**.
    - If a page **has not been modified**, it is **clean**, and eviction incurs **no write-back cost**.

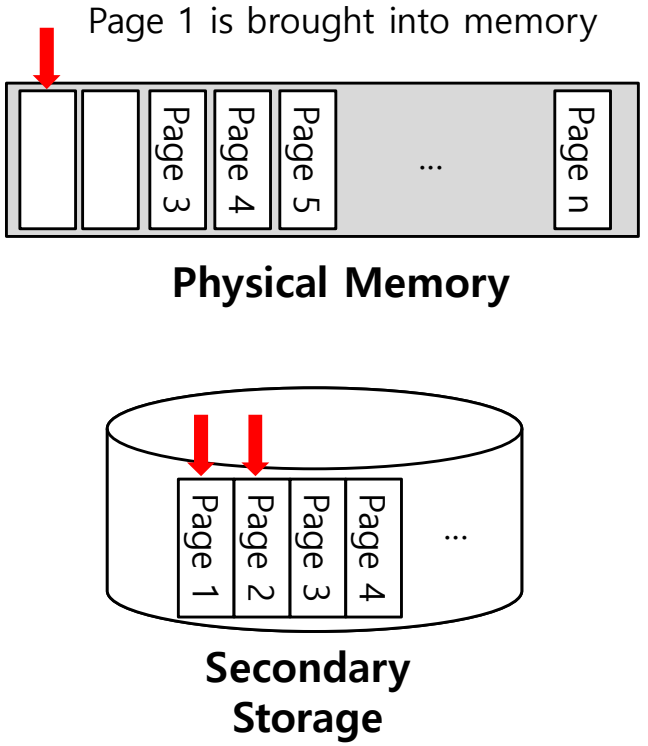
- **Incorporating Dirty Pages into the Clock Algorithm**
  - The Clock algorithm can be extended to take the dirty bit into account.
  - The replacement policy can prioritize pages in the following order:
    - **Unused and clean pages** (use bit = 0, dirty bit = 0)
    - **Unused but dirty pages** (use bit = 0, dirty bit = 1)
    - Pages that have been recently used (use bit = 1), after clearing the use bit
  - This approach reduces expensive disk write-backs by evicting clean pages first whenever possible.

Important Insight

- Evicting a clean page is cheaper than evicting a dirty page, so page replacement policies should consider write-back cost, not just recency.

Prefetching

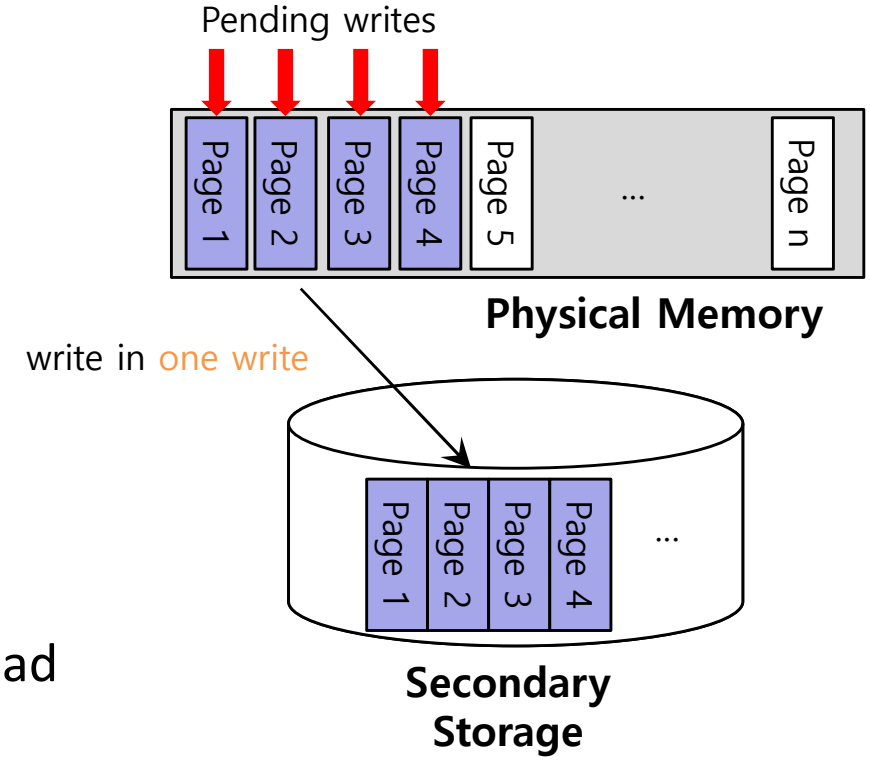
- **Prefetching**
  - The operating system predicts that a page will be needed soon and proactively brings it into memory ahead of time.
- **Example: Sequential Prefetching**
  - When Page 1 is brought into physical memory,
  - The OS predicts that Page 2 is likely to be accessed next,
  - And therefore prefetches Page 2 from secondary storage into memory.
- **Key Idea**
  - Prefetching overlaps I/O latency with computation by fetching data before it is explicitly requested.



Page 2 likely soon be accessed and thus should be brought into memory too

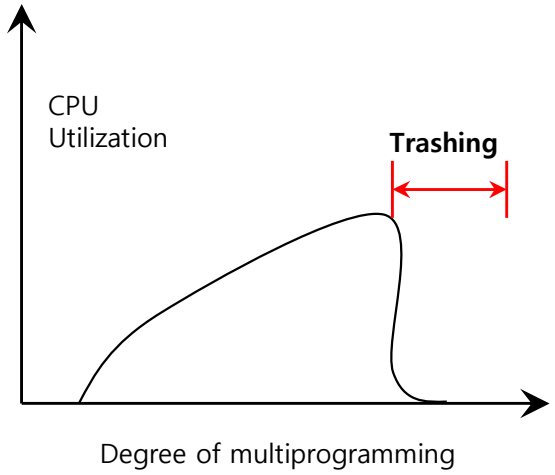
Clustering and Grouping

- **Clustering and Grouping**
  - Collect multiple pending writes in memory and write them to disk in a single operation.
    - Performing **one large write** is significantly more efficient than performing **many small writes**.
- **Key Idea**
  - Clustering improves I/O efficiency by amortizing disk access overhead across multiple writes.
- **Why Clustering Helps**
  - Reduces:
    - Disk seek and command overhead
    - I/O scheduling and interrupt overhead
  - Improves:
    - Disk bandwidth utilization
    - Overall write throughput



Thrashing

- **Thrashing**
  - Thrashing occurs when memory is oversubscribed and the combined working sets of running processes exceed the available physical memory.
    - The system spends most of its time **paging** rather than executing useful work.



- **Consequences**
  - High page fault rate
  - Low CPU utilization, despite many runnable processes
  - Performance degrades as the system becomes dominated by memory I/O
- **System Response**
  - Reduce the degree of multiprogramming:
    - Temporarily suspend or swap out a subset of processes.
    - Allow the **working sets of the remaining processes** to fit in memory.
- **Key Insight**
  - More concurrency does not always improve performance when memory is the bottleneck.